

Gmail over IMAP: what we tested, and what it proved

test-mailbox@example.com · 10 September 2026 · 43 capability tests + 16 document claims · every figure measured, none quoted

A comparison document says what someone believes a protocol can do. These two scripts run the protocol and report what happened. They took 51 seconds against a live mailbox of 7,832 messages, changed nothing, and found six defects in our own code.

38 capabilities that work	4 partial	3 genuinely missing	6 bugs the tests found
------------------------------	--------------	------------------------	---------------------------

The one thing to remember

We did not choose *IMAP*. We chose an **app password** — and one app password opens two doors:

```
imap.gmail.com:993 → reading
smtp.gmail.com:587 → sending
```

Same credential, same account. No security audit, no annual fee, no approval wait. Almost every claim that IMAP "cannot" do something is a claim about plain IMAP — and Gmail's IMAP is not plain IMAP. It advertises a capability called `X-GM-EXT-1`, which brings real labels, real thread ids, and the entire Gmail search language.

The headline for the meeting. Everything the POC needs works over the free route: the Primary tab (1,748 messages), every Gmail label, Spam, the Important marker, applying and removing labels, marking read, live polling, and sending — measured at 4.9 seconds for a real delivered message. Three things are genuinely missing, and only one of them costs us anything.

What is genuinely missing

- missing

Server-pushed notification. Gmail's API can tell us the instant mail arrives. IMAP has `IDLE`, which is close, but it holds one open socket per mailbox — a thousand users means a thousand held sockets. We poll every 20 seconds instead.
cost: up to 20 seconds of delay. For a POC, none.
- missing

Snooze. A Gmail-only concept with no IMAP equivalent.
cost: none — it is not in any use case.

missing

Forward. SMTP is wired and working; the `forward()` function itself is simply not written yet.
cost: a morning's work, not a blocker.

Test 1 • The capability audit

43 tests across nine areas. Read-only by default; the tests that write are off unless you ask for them, act on exactly one message, and undo themselves.

```
python audit_imap.py
python audit_imap.py --write // also tests writes, then reverts them
```

1 • What the server advertises

yes Speaks Gmail's IMAP dialect
X-GM-EXT-1 – this is the flag everything else depends on

yes IDLE, MOVE, UIDPLUS, CONDSTORE, QUOTA
all five advertised

2 • Can it see every Gmail label?

yes List every folder and label
19 total

yes Gmail's own system folders
All Mail • Drafts • Important • Sent Mail • Spam • Starred • Trash

yes Your own labels
10 – Custodian, Custodian.Flights, Custodian.Invoices, Custodian.Meetings ...

3 • The Gmail tabs

1,748 yes category:primary — the question sir asked

2,834 yes category:social

1,260 yes category:promotions

3,558 yes category:updates

0 empty category:forums — the query works; this mailbox has none

4 • Spam and the special folders

yes Spam
[Gmail]/Spam – 1 message

yes Sent
[Gmail]/Sent Mail – 33

yes	All Mail [Gmail]/All Mail – 7,877
yes	Trash, Starred both open

5 • What we can read about one message

yes	Its Gmail labels X-GM-LABELS → Custodian/Machine mail
yes	Its thread id X-GM-THRID 1875946006072998074
yes	Read the body WITHOUT marking it read BODY.PEEK – unread stayed True. A promise we make, so it is tested.
yes	Fetch attachments

6 • Live — can it notice new mail?

yes	Take a bookmark, poll for anything newer, bookmark survives UID 7870 → 7870. Gmail hands out a history id; IMAP has none, so we use the highest UID – which only ever increases and never repeats. Same promise, different mechanism.
no	Server-pushed notification IDLE is advertised but costs a held socket per mailbox – we poll every 20s

7 • Search

54	yes	newer_than:7d
7,493	yes	is:unread
24	yes	has:attachment
5,186	yes	from:linkedin.com
43	yes	category:primary newer_than:30d

8 • Actions that change the mailbox

yes	Apply a label appeared on the message – and it stayed in the inbox
yes	Remove a label reverted
yes	Mark read unread True → False

yes

Mark unread again
reverted

yes

The audit cleaned up after itself
probe label removed · inbox 7,832 before, 7,832 after

9 · Sending, and the remaining gaps

yes

Send a message
smtp.gmail.com – same app password

yes

Save a draft
APPEND to [Gmail]/Drafts – no SMTP needed, cannot reach anyone

partial

Forward
SMTP is wired; forward() not built yet

partial

See labels you removed in Gmail
no history feed – we diff our own footprint instead. See page 4.

partial

Rescue from spam
possible by MOVE, not built

no

Snooze
Gmail-only concept

Test 2 • The comparison document, claim by claim

Sixteen claims from the handwritten IMAP vs API notes, each one run against the live mailbox.

```
python verify_doc.py
python verify_doc.py --send // also sends one real mail to yourself
```

6

confirmed

3

wrong

3

partly right

4

policy, not testable

What the document says we CAN do — all confirmed

true

Read the entire mailbox
7,832 messages

true

Full read and write
X-GM-LABELS writes verified

true

Move messages between folders
UID COPY works — though on Gmail we add a label instead, which is better: the message does not leave the inbox

true

Send mail
delivered via smtp.gmail.com:587 in 4.9s, copy filed in Sent

true

Can read old emails — all of them

The document was right and I was wrong. It lists "send mail" under what the app-password route can do. I had said IMAP cannot send. That is true about the protocol and misleading about the product — the app password opens SMTP too, and sending now works.

What it says we CANNOT do — three of the four are wrong

false

"Will not get labels"
X-GM-LABELS returned Custodian/Machine mail on a real message; 40 messages carry user labels

false

"Will not get categories / Primary / Promo"
primary 1,748 • promotions 1,260 • social 2,834 • updates 3,558

false

"Will not get the Important marker"
is:important 187 • is:starred 1

partly

"Cannot know when mail arrives"
no push — but polling works, and the server advertises IDLE. "Cannot know" is too strong.

The duplicate claim — real, and already handled

The document notes that Gmail can put three labels on one message, so in an older connection the same message appears three or four times. That is a genuine trap, and it only springs when you scan every folder.

partly

"The same message appears 3–4 times"

in a 300-message sample of All Mail: 0 repeated Message-IDs. We scan INBOX and key on Message-ID, so a repeat collapses to one row.

The daily limits — recomputed on this mailbox

These figures depend entirely on how big an average message is, which is a property of *this mailbox* and not of Gmail. So they were measured rather than repeated.

partly

"Headers only: 500,000–600,000 msg/day"

median header 5,836 bytes → 449,184/day. 11% below the stated low end.

true

"Full messages with body: 33,000 msg/day"

median message 76,509 bytes → 34,263/day. At the stated figure.

Read this one carefully. The full-message figure is sampled from the 25 newest messages and moves with the sample — an earlier run measured 105,541 bytes and gave 24,838/day. The document's method is sound; the answer swings by a third depending on what arrived that week. If this number ever has to be defended, sample a few hundred, not 25.

The API page — policy, not protocol

n/a

\$540–3,000/yr security audit; repeats annually; 2–3 months for approval; 100 users and 7-day tokens before approval

All matches Google's published CASA tiers. Cannot be tested from code.

The detail that matters most. Every one of those costs applies to the **OAuth consent screen**. An app password has no audit, no expiry, and no 100-user cap. That is the whole reason this route exists.

What the tests found in our own code

Six defects. Every one of them looked correct from the terminal, which is the point of writing tests that talk to a real server.

1 · Applying a label was moving the message out of the inbox. The old code did COPY, then `+FLAGS \Deleted`, then EXPUNGE. On Gmail that is not filing — it is archiving. `sort_mailbox.py` promises in writing that it "only applies labels; never moves, archives or deletes", and it was archiving every message it touched. **This happened once for real during the first audit run** — one message left the inbox. It was restored immediately, the probe folder was deleted, and the count verified: 7,831 before, 7,831 after. Labels now use Gmail's own `X-GM-LABELS` and the message stays put.

2 · Sequence numbers where UIDs belonged. `list_ids()` returned a message's *position* in the folder; `gmail_search()` returned UIDs. Both were stored as `provider_id`. A position shifts whenever anything above it arrives or leaves — so an id saved today points at a different email tomorrow, and nothing raises an error when it does. The whole connector now uses `UID` commands.

3 · The label parser dropped any label without a space. Gmail quotes a label that contains a space and sends a bare one otherwise. The old pattern matched quoted strings, or bare runs of `\w` — which does not include a slash. So "Custodian/Machine mail" read fine and Custodian/AuditProbe vanished. The audit reported "label did not appear" three times while the server was replying, in the same breath, that the label was there. **Writing was never broken. Reading it back was.**

4 · `list_spam()` always returned empty. Gmail's folder is `[Gmail]/Spam`; we were looking for "Spam", "Junk" and "Bulk Mail". An empty answer from a spam check reads exactly like "no spam" — the answer you least want to be wrong about.

5 · Gmail was missing from the folder map. Trash fell through to a default of `"Trash"`, which does not exist on Gmail.

6 · The audit claimed "nothing was left changed" without checking. It printed that line unconditionally — including on the run where it had moved a message. It now counts the inbox before and after and prints the comparison, and it deletes the probe label it creates.

The label-removal gap, and how it was closed

UC-29 says that dragging a message out of one of our labels is a correction — and it happens in the user's mail app, not ours. Gmail's API can be asked "what changed since X" and will name every label removed. IMAP has no such feed; it can only ever answer "what is true now".

But we never needed a change feed. **We know exactly which messages we labelled, because we wrote it down when we did it.** So instead of listening for events, we re-read the labels on our own footprint — tens of messages a day — and look for ours going missing.

A feed tells you what changed. A short list you already own can simply be checked again.

`correct.py` never learns which door it is behind: the signature is identical on both connectors.

Run all of it yourself

Nothing here needs setup beyond the two environment variables already on this machine. Open a terminal and go.

Step 1 — get to the right folder

```
cd c:\mob_ai\poc
```

Step 2 — the capability audit (safe, changes nothing)

```
python audit_imap.py
```

Takes about 25 seconds. You will see nine sections and 43 rows marked [YES], [NO], [PART] or [SKIP] . The last line counts the inbox before and after and tells you whether anything moved.

Step 3 — the same audit, including writes

```
python audit_imap.py --write
```

Now the five [SKIP] rows in section 8 run for real: it labels one message, checks the label arrived, removes it, marks the message read, marks it unread again, and deletes the label it created. **Every write is undone in a finally block**, and the closing line proves the inbox count did not move.

Step 4 — verify the comparison document

```
python verify_doc.py
```

Sixteen claims, each marked [TRUE], [FALSE], [PARTLY] or [n/a] . It measures 25 real messages to recompute the daily limits, so it takes about 30 seconds.

Step 5 — prove sending actually works

```
python verify_doc.py --send
```

This sends **one real email to your own address** through smtp.gmail.com. Check your inbox for "**Custodian — SMTP proof**", and check Sent Mail for the filed copy. It goes nowhere else and to nobody else.

Step 6 — see it all in a browser

```
python web.py --watch  
python web.py --watch --imap // same page, free door
```

Opens the mailbox view at `localhost:8000` and runs the agent in the background at the same time. The pill at the top right names the door you came in through — watch it change between the two commands while

the list stays identical.

If it says the variables are not set. They were added with `setx`, which only affects terminals opened *afterwards*. Close the terminal, open a new one, and run it again.

What each result marker means

yes / true

tested against the live server and it worked

part / partly

works with a caveat, or works by a different mechanism than the document assumed

no / false

genuinely does not work, or the claim is contradicted by measurement

skip / n/a

not run — either a write test without `--write`, or a policy claim that no code can test

For the meeting

If asked: can it read all the Gmail labels?

Yes — all nineteen. Every one of your own labels, every Gmail system folder including Spam, and all four category tabs. Primary is 1,748 messages, measured over IMAP.

If asked: do the actions still show up in Gmail?

Yes, immediately — because we use Gmail's own X-GM-LABELS, the same mechanism the API uses. Applying a label, removing it, marking read and marking unread were all tested and all reverted cleanly.

If asked: what are we sacrificing?

Of 43 capabilities, 38 work. Three are genuinely missing: push notification, which costs us up to twenty seconds of delay; snooze, which no use case needs; and forward, which is a morning's work because SMTP is already wired. Nothing on that list blocks the POC.

If asked: is the comparison document wrong?

It is right about the protocol and wrong about the product. Plain IMAP genuinely has no labels, no categories and no Important marker. Gmail's IMAP is not plain IMAP — it advertises X-GM-EXT-1, and those three claims fail against a live server. The document is also right about something I had got wrong: the app-password route can send.

If asked: how do we know any of this?

Two scripts, fifty-one seconds, on a real mailbox. They found six bugs in our own code — including one where applying a label was quietly taking the message out of the inbox. From the terminal it had looked correct the whole time.

The line worth ending on. A comparison document tells you what someone believes a protocol can do. We ran it instead — and the difference between those two things was six defects, one of which was moving customers' mail.